Nama : Elsa Marthalinda

NIM : 254107020204

Kelas : TI – 1G

LAPORAN JOBHSEET 10

2.1 Percobaan 1 : Operasi Dasar Queue

Kode Program **Queue7.java;**

```
package P1Jobsheet10;
public class Queue7 {
    int[] data;
    int front;
    int rear;
    int size;
    int max;

    public Queue7(int n) {
        max = n;
        data = new int[max];
        size = 0;
        front = rear = -1;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
```

```
        return true;
    } else {
        return false;
    }
}

public void peek() {
    if (!IsEmpty()) {
        System.out.println("Elemen terdepan: " + data[front]);
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void print() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.print(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
```

```
        System.out.println("Queue masih kosong");
    }
}

public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println("Queue sudah penuh");
    } else {
        if (IsEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}

public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        dt = data[front];
        size--;
        if (IsEmpty()) {
            front = rear = -1;
        } else {
            if (front == max - 1) {
                front = 0;
```

```
            } else {
                front++;
            }
        }
    }
    return dt;
  }
}
```

Kode Program **QueueMain.java;**

```
package P1Jobsheet10;
import java.util.Scanner;
public class QueueMain7 {

  public static void menu() {
    System.out.println("Masukkan operasi yang diinginkan:");
    System.out.println("1. Enqueue");
    System.out.println("2. Dequeue");
    System.out.println("3. Print");
    System.out.println("4. Peek");
    System.out.println("5. Clear");
    System.out.println("---------------------");
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Masukkan kapasitas queue: ");
    int n = sc.nextInt();

    Queue7 Q = new Queue7(n);
    int pilih;

    do {
```

```
            menu();
            pilih = sc.nextInt();

            switch (pilih) {
                case 1:
                    System.out.print("Masukkan data baru: ");
                    int dataMasuk = sc.nextInt();
                    Q.Enqueue(dataMasuk);
                    break;

                case 2:
                    int dataKeluar = Q.Dequeue();
                    if (dataKeluar != 0) {
                        System.out.println("Data yang dikeluarkan: " + dataKeluar);
                    }
                    break;

                case 3:
                    Q.print();
                    break;

                case 4:
                    Q.peek();
                    break;

                case 5:
                    Q.clear();
                    break;
            }

        } while (pilih >= 1 && pilih <= 5);
    }
}
```

Hasil Output **QueueMain7.java;**

```
Masukkan kapasitas queue: 4
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
1
Masukkan data baru: 31
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
4
Elemen terdepan: 15
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
```

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?
   - Nilai awal front dan rear diatur menjadi -1 karena digunakan sebagai penanda bahwa queue masih dalam kondisi kosong dan belum memiliki elemen sama sekali. Nilai -1 menunjukkan bahwa kedua variabel tersebut belum menunjuk ke indeks mana pun pada array. Sementara itu, variabel size diinisialisasi dengan nilai 0 karena memang belum ada data yang tersimpan di dalam queue, sehingga jumlah elemennya masih nol.
2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {
    rear = 0;
```

   - Potongan kode pada method Enqueue yang memeriksa kondisi **rear == max - 1** bertujuan untuk mengetahui apakah posisi rear sudah berada di indeks terakhir array.
3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {
    front = 0;
```

- Pada method **front == max - 1**, jika kondisi ini benar, maka nilai front akan dikembalikan ke indeks 0. Tujuan dari logika ini adalah untuk menjaga pergerakan front tetap sesuai dengan konsep circular queue, sehingga ketika sudah mencapai ujung array, proses pengambilan data dapat dilanjutkan kembali dari awal array.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?
   - Pada method print, variabel i diinisialisasi dengan nilai front, bukan 0. Hal ini karena elemen pertama dalam queue tidak selalu berada di indeks 0, melainkan berada pada posisi yang ditunjuk oleh front. Dengan menggunakan i = front, proses pencetakan hanya akan menampilkan elemen yang benar-benar berada dalam antrian, bukan seluruh isi array.
5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

   - Potongan kode i = (i + 1) % max digunakan untuk melakukan pergeseran indeks secara melingkar. Operasi modulus (% max) memastikan bahwa jika nilai indeks melebihi batas maksimum array, maka indeks akan kembali ke posisi awal (0). Dengan demikian, perulangan tetap berjalan dengan benar dalam struktur circular queue.
6. Tunjukkan potongan kode program yang merupakan queue overflow!
   -
```
if (IsFull()) {
    System.out.println(x: "Queue sudah penuh");
```
7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!
   -
```
public void Enqueue(int dt) {
    if (IsFull()) {
        System.out.println(x: "Queue sudah penuh");
        System.exit(status: 0);
```
   -
```
public int Dequeue() {
    int dt = 0;
    if (IsEmpty()) {
        System.out.println(x: "Queue masih kosong");
        System.exit(status: 0);
```

```
Masukkan kapasitas queue: 2
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
1
Masukkan data baru: 23
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
1
Masukkan data baru: 46
Masukkan operasi yang diinginkan:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
---------------------
1
Masukkan data baru: 45
Queue sudah penuh
```

2.2. Percobaan 2 : Antrian Layanan Akademik

Kode Program **Mahasiswa7.java;**

```
package P2Jobsheet10;
public class Mahasiswa7 {
    String nim;
    String nama;
    String prodi;
    String kelas;

    public Mahasiswa7(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
}

    public void tampilkanData() {
        System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
    }
```

```
}
```

Kode Program **AntrianLayanan7.java;**

```
package P2Jobsheet10;
public class AntrianLayanan7 {
    Mahasiswa7[] data;
    int front;
    int rear;
    int size;
    int max;

    public AntrianLayanan7(int max) {
        this.max = max;
        this.data = new Mahasiswa7[max];
        this.front = 0;
        this.rear = -1;
        this.size = 0;
    }

    public boolean IsEmpty() {
        if (size == 0) {
            return true;
        } else {
            return false;
        }
    }

    public boolean IsFull() {
        if (size == max) {
            return true;
        } else {
            return false;
        }
```

```
}

public void peek() {
    if (!IsEmpty()) {
        System.out.println("Elemen terdepan: " + data[front]);
    } else {
        System.out.println("Queue masih kosong");
    }
}

public void print() {
    if (IsEmpty()) {
        System.out.println("Queue masih kosong");
    } else {
        int i = front;
        while (i != rear) {
            System.out.print(data[i] + " ");
            i = (i + 1) % max;
        }
        System.out.println(data[i] + " ");
        System.out.println("Jumlah elemen = " + size);
    }
}

public void clear() {
    if (!IsEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println("Queue berhasil dikosongkan");
    } else {
        System.out.println("Queue masih kosong");
    }
}
```

```
public void tambahAntrian(Mahasiswa7 mhs) {
    if (IsFull()) {
        System.out.println("Antrian penuh, tidak dapat menambah mahasiswa.");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + " berhasil masuk ke antrian");
}

public Mahasiswa7 layaniMahasiswa() {
    Mahasiswa7 dt = null;
    if (IsEmpty()) {
        System.out.println("Antrian kosong.");
        return null;
    }
    Mahasiswa7 mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}

public void lihatTerdepan() {
    if (!IsEmpty()) {
        System.out.println("Antrian kosong.");
    } else {
        System.out.print("Mahasiswa terdepan: ");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        data[front].tampilkanData();
    }
}

public void tampilkanSemua() {
```

```
        if (IsEmpty()) {
            System.out.println("Antrian kosong.");
            return;
        }
        System.out.println("Daftar Mahasiswa dalam Antrian:");
        System.out.println("NIM - NAMA - PRODI - KELAS");
        for (int i = 0; i < size; i++) {
            int index = (front + i) % max;
            System.out.print((i + 1) + ". ");
            data[index].tampilkanData();
        }
    }

    public int getJumlahAntrian() {
        return size;
    }
}
```

Kode Program **LayananAkademikSIAKAD7.java;**

```
package P2Jobsheet10;
import java.util.Scanner;
public class LayananAkademikSIAKAD7 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan7 antrian = new AntrianLayanan7(5);
        int pilihan;

        do {
            System.out.println("\n=== Menu Antrian Layanan Akademik ===");
            System.out.println("1. Tambah Mahasiswa ke Antrian");
            System.out.println("2. Layani Mahasiswa");
            System.out.println("3. Lihat Mahasiswa Terdepan");
```

```
System.out.println("4. Lihat Semua Antrian");
System.out.println("5. Jumlah Mahasiswa dalam Antrian");
System.out.println("0. Keluar");
System.out.print("Pilih menu: ");
pilihan = sc.nextInt(); sc.nextLine();

switch (pilihan) {
    case 1:
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Nama  : ");
        String nama = sc.nextLine();
        System.out.print("Prodi : ");
        String prodi = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();
        Mahasiswa7 mhs = new Mahasiswa7(nim, nama, prodi, kelas);
        antrian.tambahAntrian(mhs);
        break;

    case 2:
        Mahasiswa7 dilayani = antrian.layaniMahasiswa();
        if (dilayani != null) {
            System.out.print("Melayani mahasiswa: ");
            dilayani.tampilkanData();
        }
        break;

    case 3:
        antrian.lihatTerdepan();
        break;

    case 4:
        antrian.tampilkanSemua();
```

```
                break;

            case 5:
                System.out.println("Jumlah dalam antrian: " + antrian.getJumlahAntrian());
                break;

            case 0:
                System.out.println("Terima kasih.");
                break;

            default:
                System.out.println("Pilihan tidak valid.");
                break;
        }
    } while (pilihan != 0);

    sc.close();
  }
}
```

Hasil Output **LayananAkademikSIAKAD7;**

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 123
Nama  : Aldi
Prodi : TI
Kelas : 1A
Aldi berhasil masuk ke antrian

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM   : 124
Nama  : Bobi
Prodi : TI
Kelas : 1G
Bobi berhasil masuk ke antrian

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G
```

```
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Melayani mahasiswa: 123 - Aldi - TI - 1A

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam Antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - Bobi - TI - 1G

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1

=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 0
Terima kasih.
```

2.2.3 Pertanyaan

Lakukan modifikasi program dengan menambahkan method baru bernama **LihatAkhir** pada class **AntrianLayanan** yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu **6. Cek Antrian paling belakang** pada class **LayananAkademikSIAKAD** sehingga method **LihatAkhir** dapat dipanggil!

Modifikasi AntrianLayanan7;

```java
public void LihatAkhir() {
    if (IsEmpty()) {
        System.out.println(x: "Antrian kosong.");
    } else {
        System.out.println(x: "Mahasiswa paling belakang:");
        System.out.println(x: "NIM - NAMA - PRODI - KELAS");
        data[rear].tampilkanData();
    }
}
```

Modifikasi LayananAkademikSIAKAD7;

```
System.out.println(x: "6. Cek Antrian Paling Belakang");
```

```
case 6:
    antrian.LihatAkhir();
    break;
```

Hasil Output:

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian Paling Belakang
0. Keluar
Pilih menu: 6
Mahasiswa paling belakang:
NIM - NAMA - PRODI - KELAS
124 - Bobi - TI - 1G
```

2.3 Tugas

Kode Program KRSMahasiswa7.java;

```
package TugasJobsheet10;
public class KRSMahasiswa7 {
    String nim, nama, prodi, kelas;

    public KRSMahasiswa7(String nim, String nama, String prodi, String kelas) {
        this.nim = nim;
        this.nama = nama;
        this.prodi = prodi;
        this.kelas = kelas;
    }

    public void tampilkanData() {
        System.out.println(nim + " | " + nama + " | " + prodi + " | " + kelas);
```

```
    }
}
```

Kode Program AntrianKRS7.java;

```
package TugasJobsheet10;
public class AntrianKRS7 {
    KRSMahasiswa7[] data;
    int front, rear, size, max;
    int sudahProses = 0;
    int kuotaDPA = 30;

    public AntrianKRS7(int max) {
        this.max = max;
        data = new KRSMahasiswa7[max];
        front = 0;
        rear = -1;
        size = 0;
    }

    public boolean isEmpty() {
        return size == 0;
    }

    public boolean isFull() {
        return size == max;
    }

    public void clear() {
        front = rear = -1;
        size = 0;
        System.out.println("Antrian dikosongkan");
    }
```

```
public void enqueue(KRSMahasiswa7 m) {
    if (isFull()) {
        System.out.println("Antrian penuh!");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = m;
    size++;
    System.out.println(m.nama + " masuk antrian");
}

public void prosesKRS() {
    if (isEmpty()) {
        System.out.println("Antrian kosong!");
        return;
    }

    if (sudahProses >= kuotaDPA) {
        System.out.println("Kuota DPA sudah penuh (30 mahasiswa)");
        return;
    }

    System.out.println("Memproses KRS:");

    for (int i = 0; i < 2; i++) {
        if (!isEmpty() && sudahProses < kuotaDPA) {
            KRSMahasiswa7 m = data[front];
            front = (front + 1) % max;
            size--;
            sudahProses++;

            System.out.print("Diproses: ");
            m.tampilkanData();
        }
```

```
    }
}

public void tampilSemua() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
        return;
    }

    System.out.println("Daftar Antrian:");
    for (int i = 0; i < size; i++) {
        int idx = (front + i) % max;
        data[idx].tampilkanData();
    }
}

public void lihat2Terdepan() {
    if (size == 0) {
        System.out.println("Antrian kosong");
    } else {
        System.out.println("2 Antrian terdepan:");
        for (int i = 0; i < 2 && i < size; i++) {
            int idx = (front + i) % max;
            data[idx].tampilkanData();
        }
    }
}

public void lihatAkhir() {
    if (isEmpty()) {
        System.out.println("Antrian kosong");
    } else {
        System.out.println("Antrian terakhir:");
        data[rear].tampilkanData();
```

```
        }
    }

    public void CetakStatistikAntrian() {
        System.out.println("Jumlah antrian: " + size);
        System.out.println("Sudah proses KRS: " + sudahProses);
        int sisa = kuotaDPA - sudahProses;
        System.out.println("Sisa kuota proses: " + sisa);
    }
}
```

Kode Program MainKRS7.java;

```
package TugasJobsheet10;
import java.util.Scanner;
public class MainKRS7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianKRS7 antrian = new AntrianKRS7(10);
        int pilih;

        do {
            System.out.println("\n=== ANTRIAN KRS DPA ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Proses KRS (2 Mahasiswa)");
            System.out.println("3. Tampilkan Semua Antrian");
            System.out.println("4. Lihat 2 Terdepan");
            System.out.println("5. Lihat Antrian Terakhir");
            System.out.println("6. Cetak Statistik Antrian");
            System.out.println("7. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih: ");
            pilih = sc.nextInt();
            sc.nextLine();
```

```
switch (pilih) {
    case 1:
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Nama  : ");
        String nama = sc.nextLine();
        System.out.print("Prodi : ");
        String prodi = sc.nextLine();
        System.out.print("Kelas : ");
        String kelas = sc.nextLine();

        KRSMahasiswa7 m = new KRSMahasiswa7(nim, nama, prodi, kelas);
        antrian.enqueue(m);
        break;

    case 2:
        antrian.prosesKRS();
        break;

    case 3:
        antrian.tampilSemua();
        break;

    case 4:
        antrian.lihat2Terdepan();
        break;

    case 5:
        antrian.lihatAkhir();
        break;

    case 6:
        antrian.CetakStatistikAntrian();
```

```
                break;

            case 7:
                antrian.clear();
                break;

            case 0:
                System.out.println("Terima kasih");
                break;

            default:
                System.out.println("Menu tidak valid");
        }

    } while (pilih != 0);
    sc.close();
  }
}
```

Hasil Output

```
=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 123
Nama  : Aldi
Prodi : TI
Kelas : 1A
```

```
Aldi masuk antrian

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 124
Nama  : Bobi
Prodi : TI
Kelas : 1B
Bobi masuk antrian

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 125
Nama  : Deswita
Prodi : TI
Kelas : 1C
Deswita masuk antrian

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
```

```
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 126
Nama  : Hafidza
Prodi : TI
Kelas : 1D
Hafidza masuk antrian

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 1
NIM   : 127
Nama  : Khania
Prodi : TI
Kelas : 1E
Khania masuk antrian

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
```

```
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 2
Memproses KRS:
Diproses: 123 | Aldi | TI | 1A
Diproses: 124 | Bobi | TI | 1B

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 3
Daftar Antrian:
125 | Deswita | TI | 1C
126 | Hafidza | TI | 1D
127 | Khania | TI | 1E

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 4
2 Antrian terdepan:
125 | Deswita | TI | 1C
126 | Hafidza | TI | 1D
```

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 5
Antrian terakhir:
127 | Khania | TI | 1E

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 6
Jumlah antrian: 3
Sudah proses KRS: 2
Sisa kuota proses: 28

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian

```
0. Keluar
Pilih: 7
Antrian dikosongkan

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 3
Antrian kosong

=== ANTRIAN KRS DPA ===
1. Tambah Antrian
2. Proses KRS (2 Mahasiswa)
3. Tampilkan Semua Antrian
4. Lihat 2 Terdepan
5. Lihat Antrian Terakhir
6. Cetak Statistik Antrian
7. Kosongkan Antrian
0. Keluar
Pilih: 0
Terima kasih
```